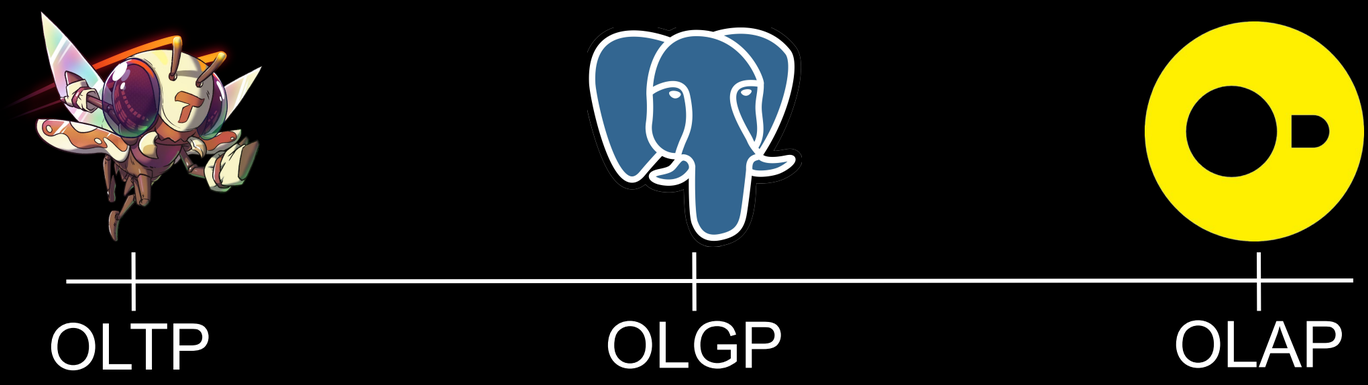
The Fastest Object Storage Client in the World



TigerBeetle: Transaction Processing (OLTP)

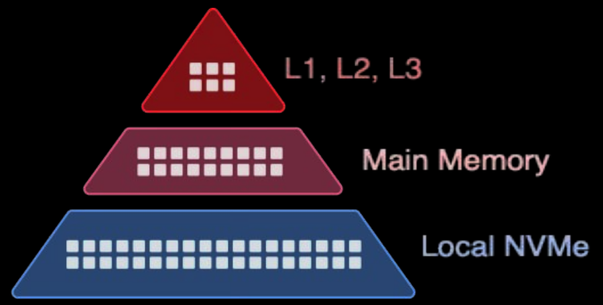
Financial transactions database built for safety and 1000x performance.

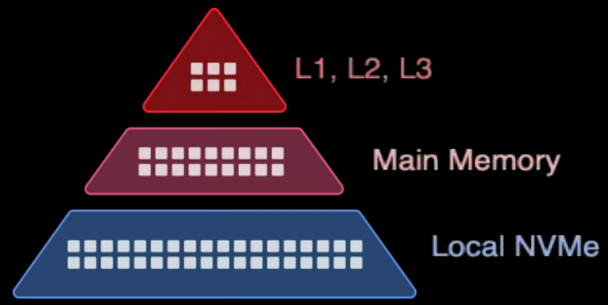
- Designed for double-entry accounting and high throughput
- Debit/credit interface instead of SQL
- Strict Serializability
- RSM on top of an LSM

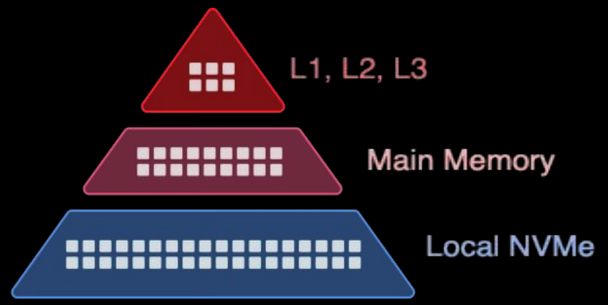


TigerBeetle: Replicated State Machine

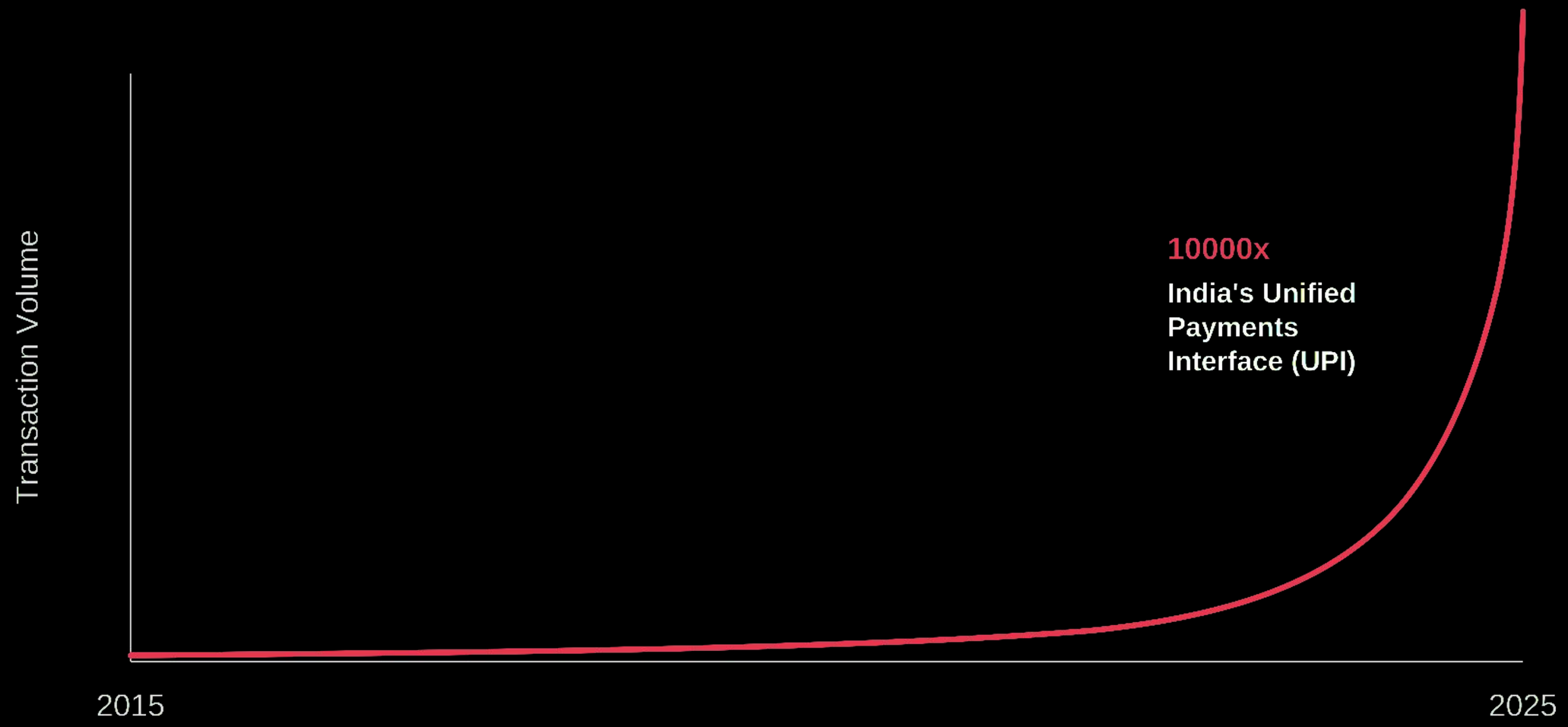
- State is replicated across nodes in a cluster
- Log Structured Merge Tree to store data
  - Exploits memory hierarchy







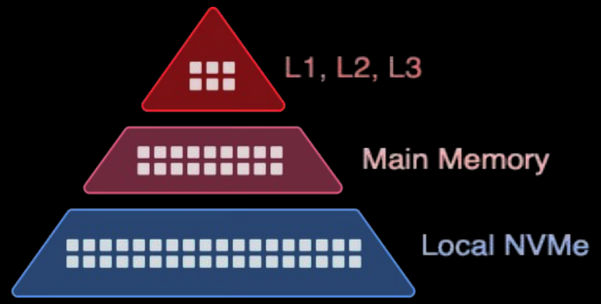
100x 1000x 10000x

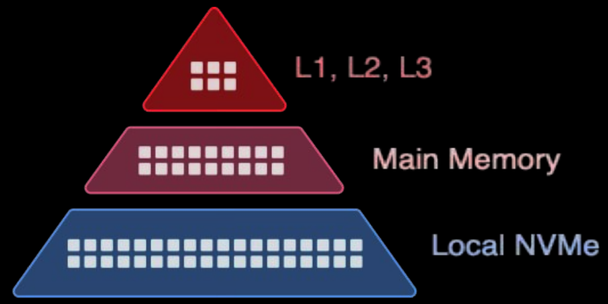


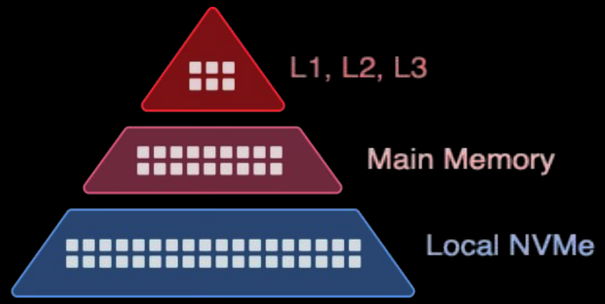
Source: https://www.rbi.org.in/scripts/publications.aspx

TigerBeetle: Replicated State Machine

- State is replicated across nodes in a cluster
- Log Structured Merge Tree to store data
  - Exploits memory hierarchy
  - Problem: Bounded by size of Local NVMe, Recovery Time

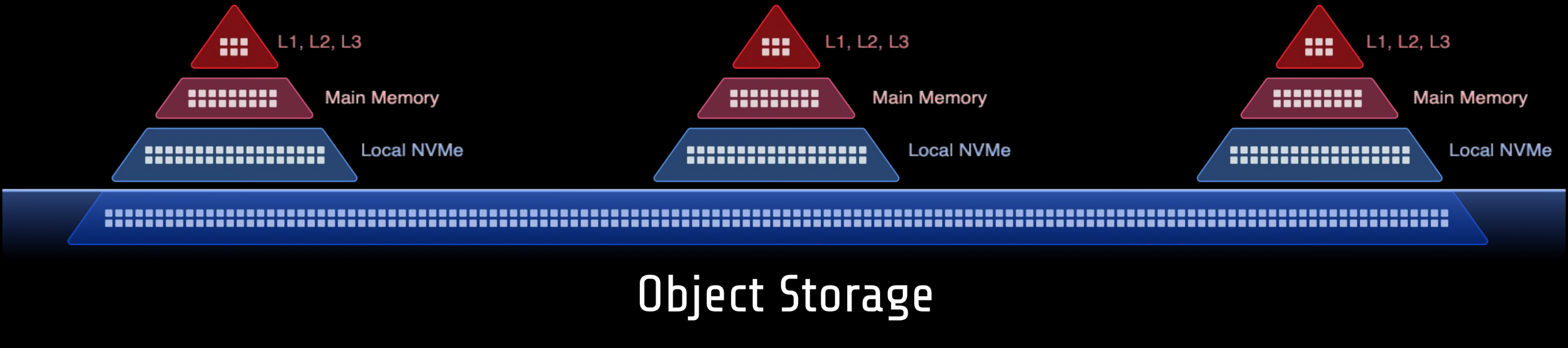






TigerBeetle: Diagonal Scaling

- Replicated State Machine
- Log Structured Merge Tree
- Object Storage



Object Storage SDKs



- Are often careless with blocking syscalls
  - Default to blocking DNS resolution (getaddrinfo)
- Spawn their own thread pools for async work
- Use many other libraries internally
  - AWS-SDK-CPP has >10 dependencies
- Allocate, deallocate, copy at runtime

TIGER STYLE

Safety

Performance

Experience



The best time to solve performance, **to get the huge 1000x wins**, is in the design phase.

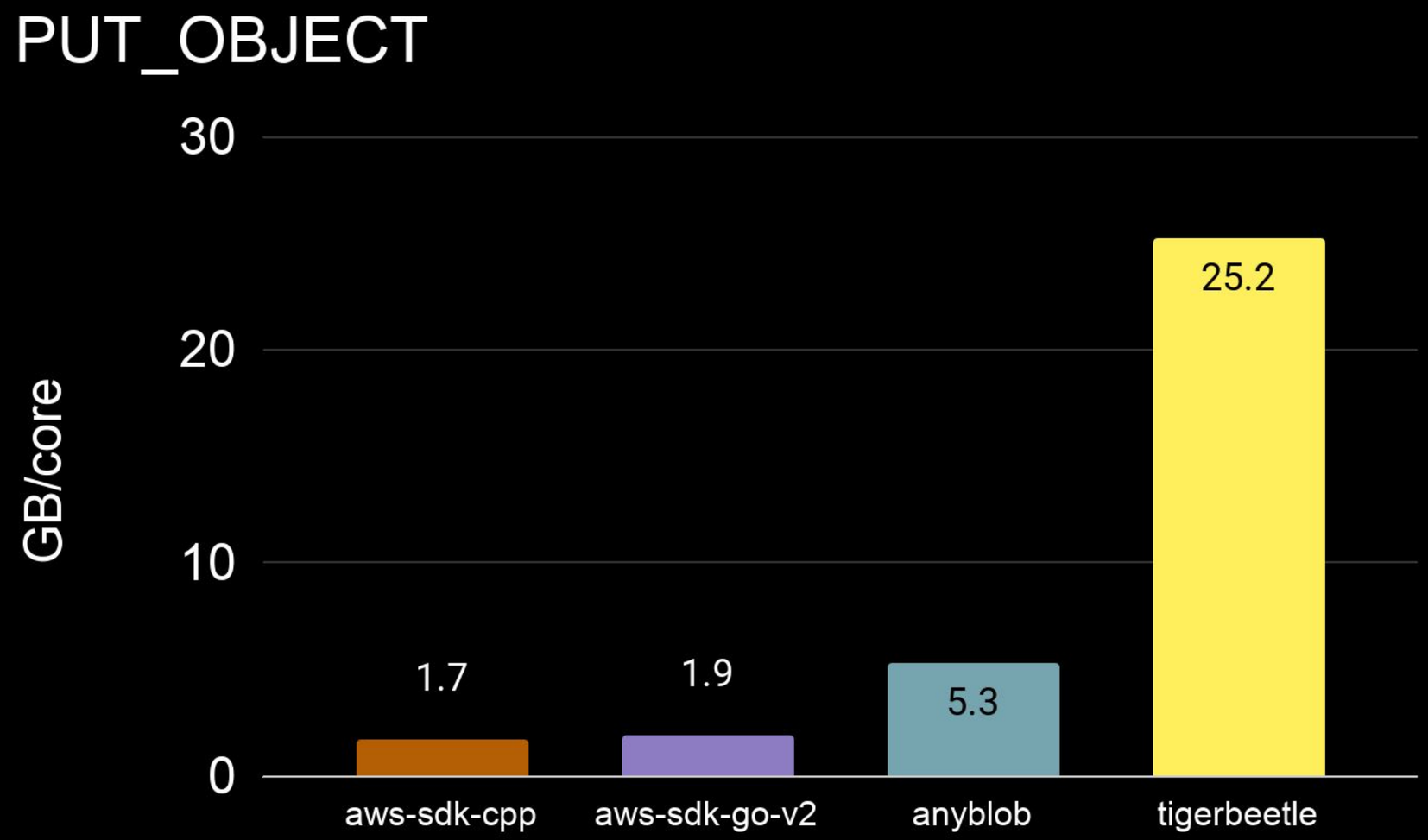
Perform back-of-the-envelope sketches.

All memory must be statically allocated at startup.

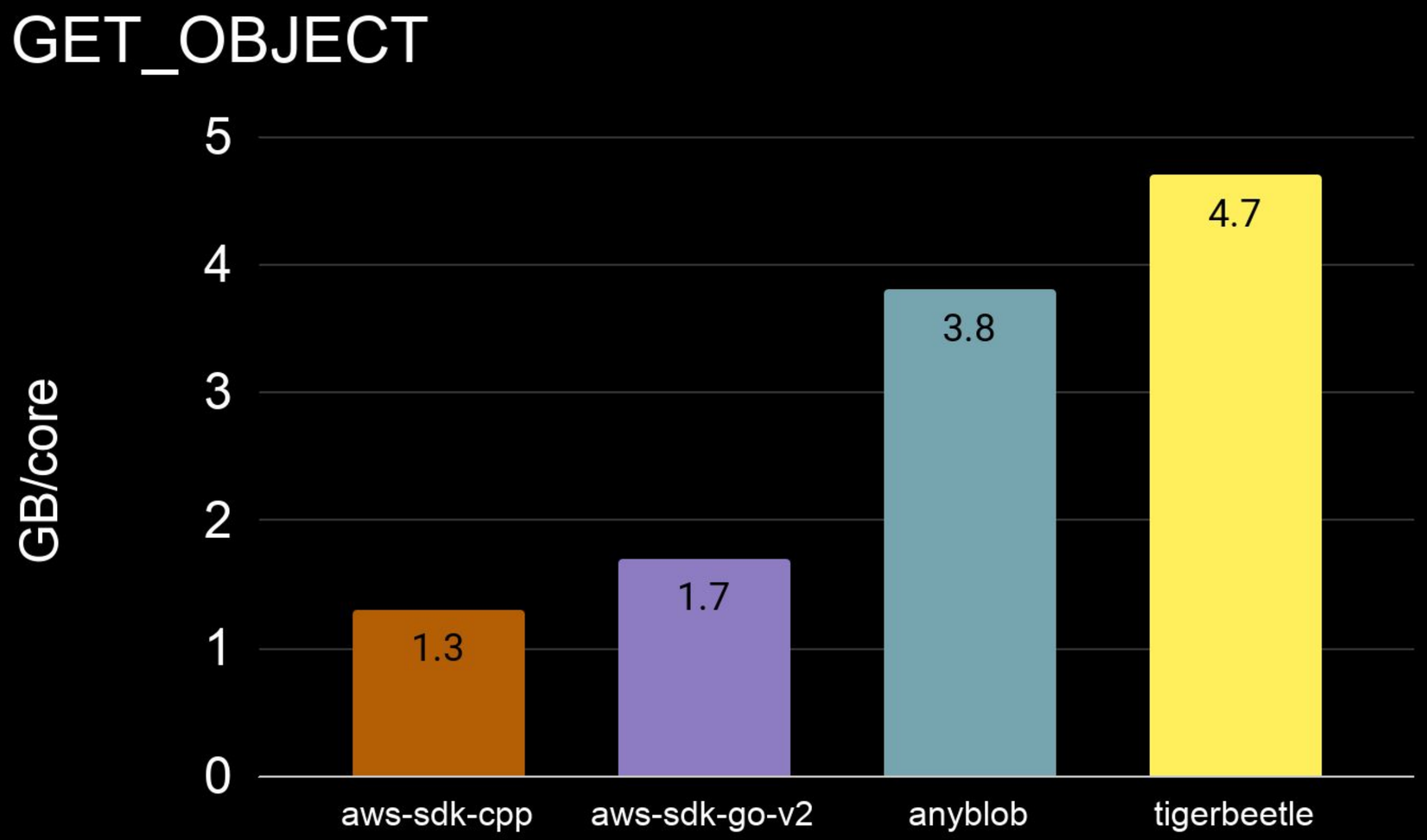
Don't react directly to external events. Your Program should run at its own pace.

Zero dependencies, apart from the Zig toolchain.

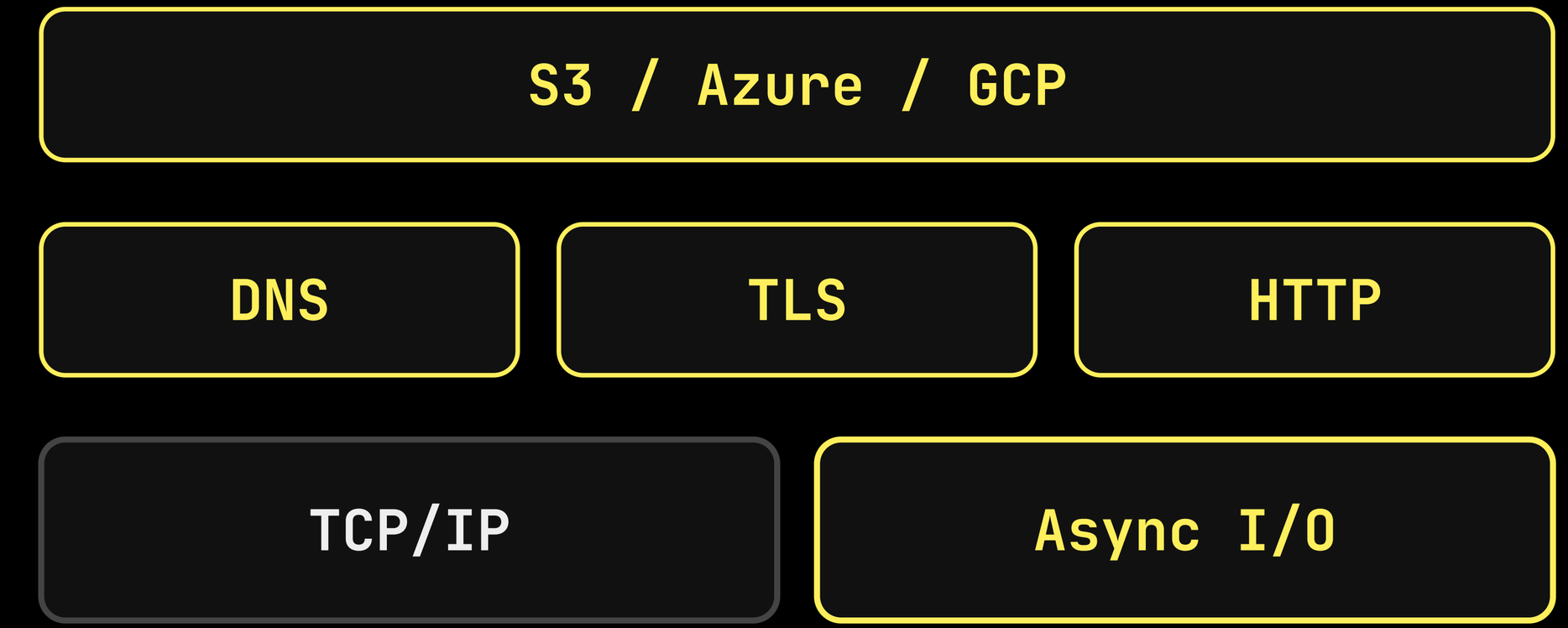
The Fastest Object Storage Client in the World



The Fastest Object Storage Client in the World



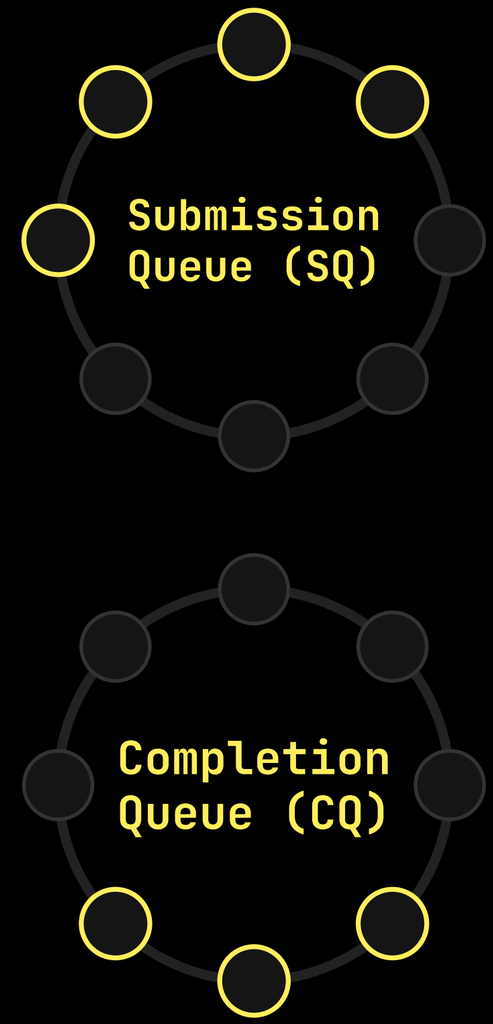
Object Storage Client Stack



Async I/O: IO_URING

Async I/O

- All network and file I/O goes through `IO_URING`
- Every component is parametrized over I/O
  - `fn ComponentType(comptime IO: type)`
- Mock I/O: same interface, backed by a seeded PRNG
  - Injectable connect / receive / send failures
  - Partial reads, short writes, bitflips
- Optimizations: Batching, Zero-Copy, Registered Buffers, Flags



Async I/O: IO_URING

- All netw
- Every c
  - fn C
- Mock I/
  - Injec
  - Part
- Optimiz

High-Performance DBMSs with io_uring: When and How to Use It

Matthias Jasny
Technische Universität Darmstadt
matthias.jasny@tu-darmstadt.de

Muhammad El-Hindi
Technische Universität München
muhammad.el-hindi@tum.de

Tobias Ziegler
TigerBeetle
tobias@tigerbeetle.com

Viktor Leis
Technische Universität München
viktor.leis@tum.de

Carsten Binnig
Technische Universität Darmstadt &
DFKI & hessian.AI
carsten.binnig@tu-darmstadt.de

ABSTRACT

We study how modern database systems can leverage the Linux io_uring interface for efficient, low-overhead I/O. io_uring is an asynchronous system call batching interface that unifies storage and network operations, addressing limitations of existing Linux I/O interfaces. However, naively replacing traditional I/O interfaces with io_uring does not necessarily yield performance benefits. To demonstrate when io_uring delivers the greatest benefits and how to use it effectively in modern database systems, we evaluate it in two use cases: Integrating io_uring into a storage-bound buffer manager and using it for high-throughput data shuffling in network-bound analytical workloads. We further analyze how advanced

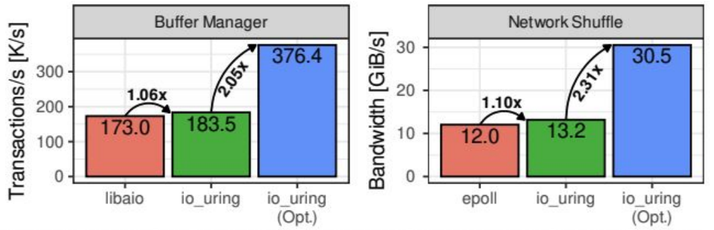
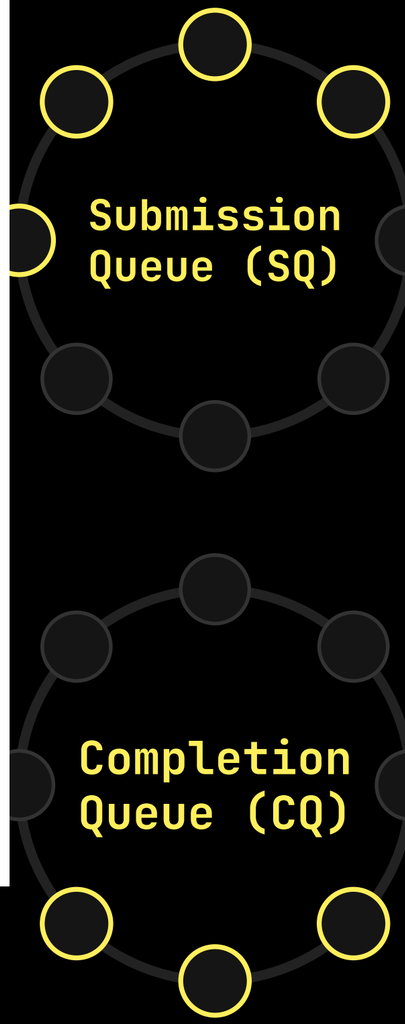


Figure 1: Performance comparison between traditional I/O interfaces and io_uring in a buffer manager and network shuffle. Naive use only yields modest gains, whereas designs that fully exploit io_uring more than double the performance.

Async I/O

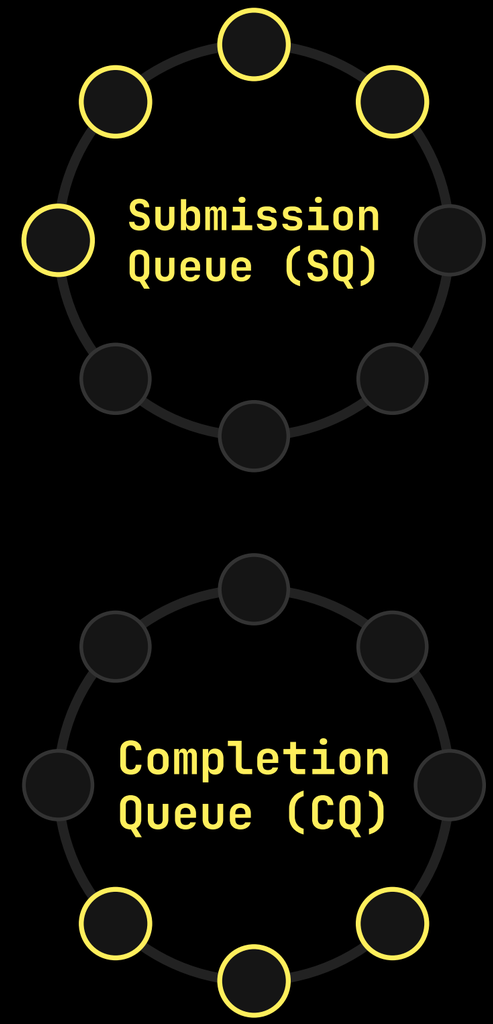
User | Kernel



Async I/O: IO_URING

Async I/O

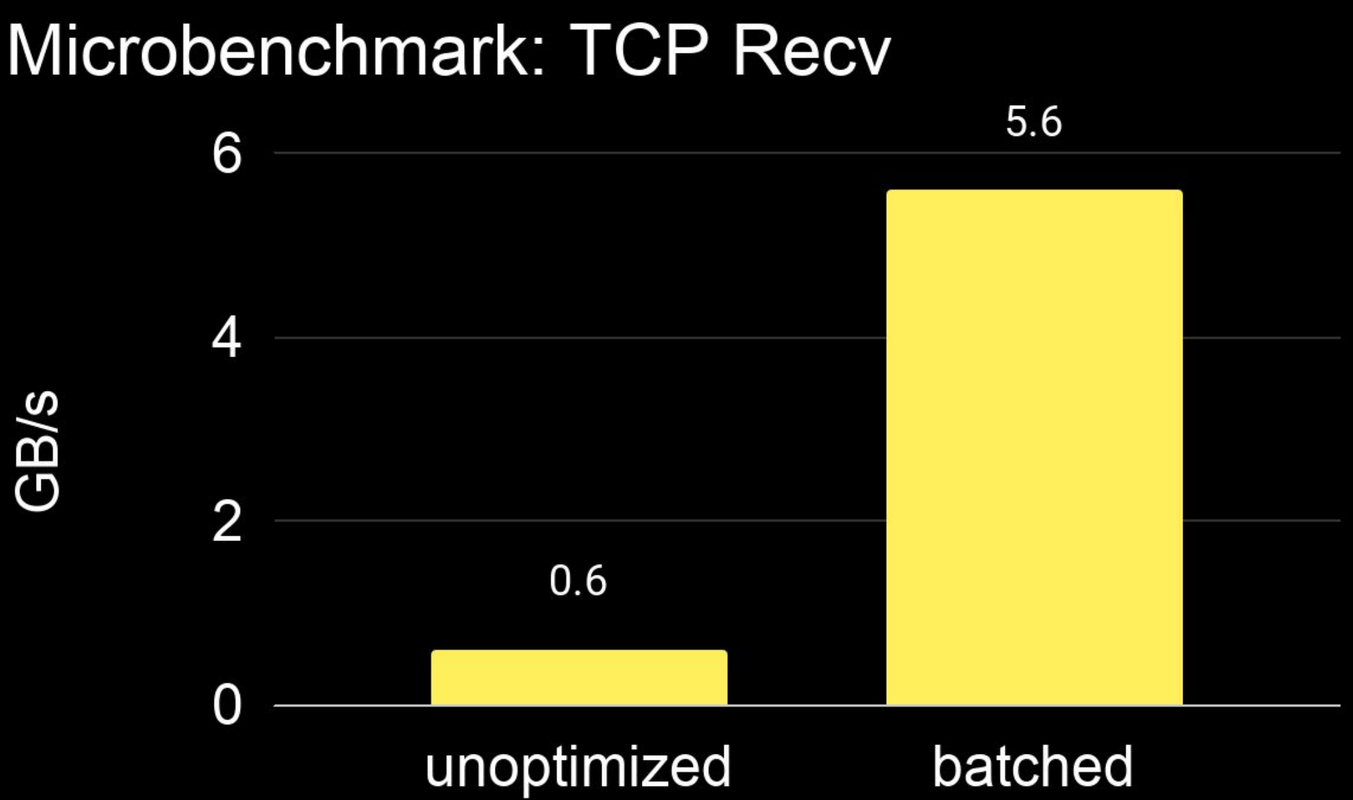
- All network and file I/O goes through `IO_URING`
- Every component is parametrized over I/O
  - `fn ComponentType(comptime IO: type)`
- Mock I/O: same interface, backed by a seeded PRNG
  - Injectable connect / receive / send failures
  - Partial reads, short writes, bitflips
- Optimizations: Batching, Zero-Copy, Registered Buffers, Flags

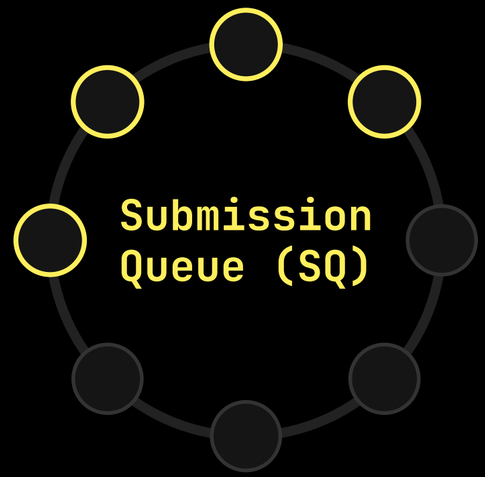


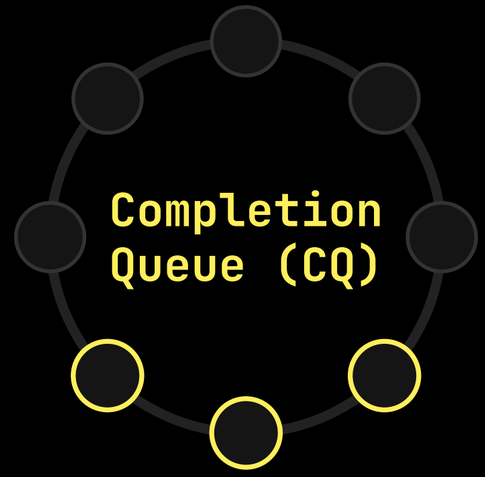
Async I/O: IO_URING

Async I/O

User | Kernel

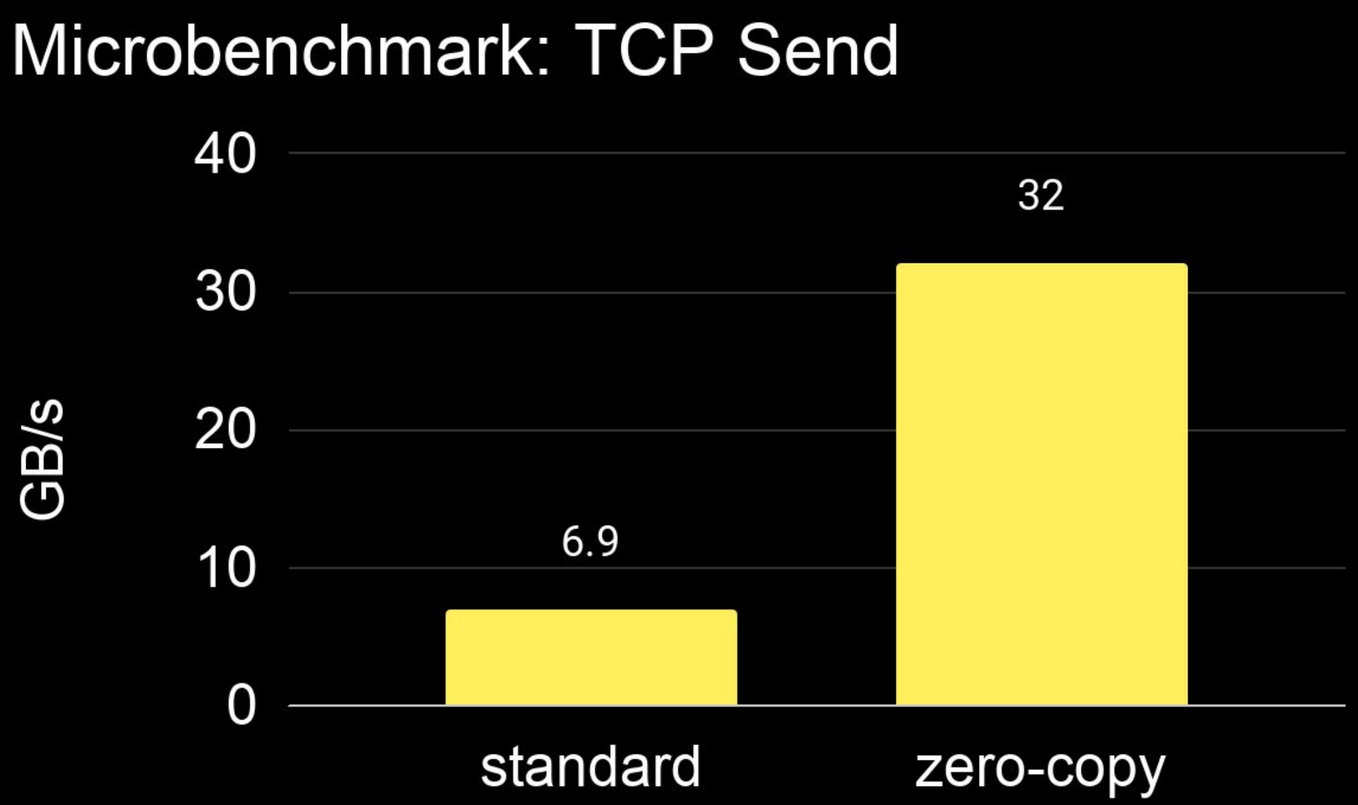


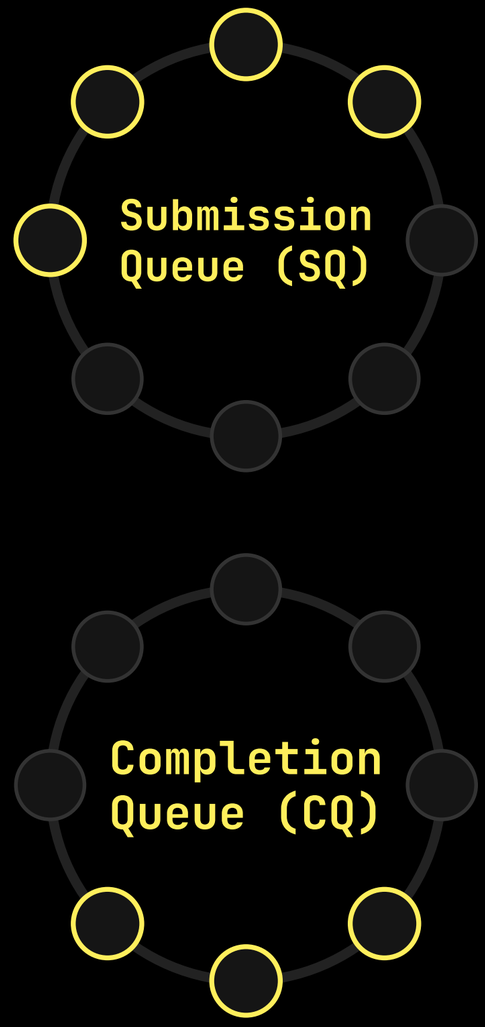




Async I/O: IO_URING

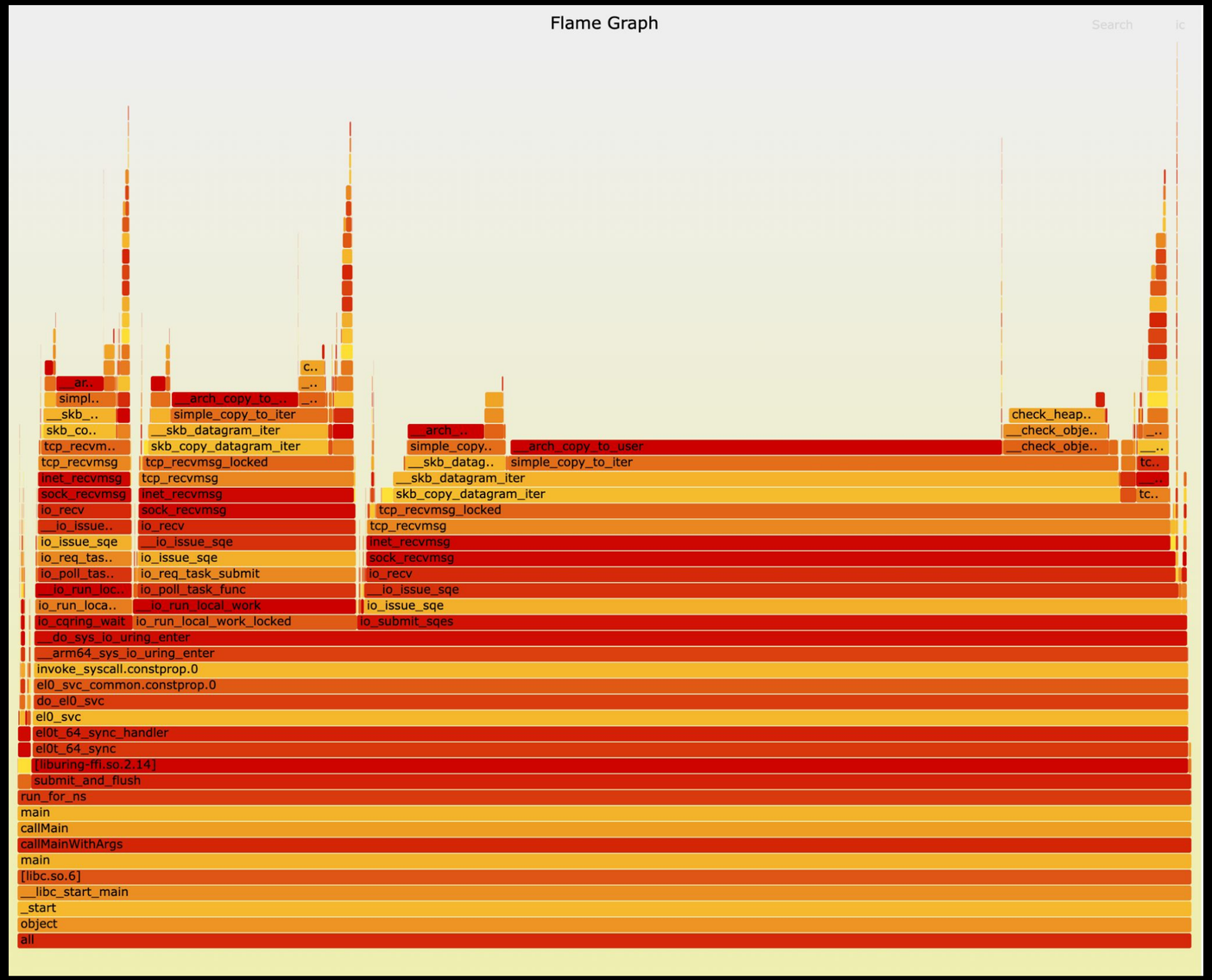
Async I/O



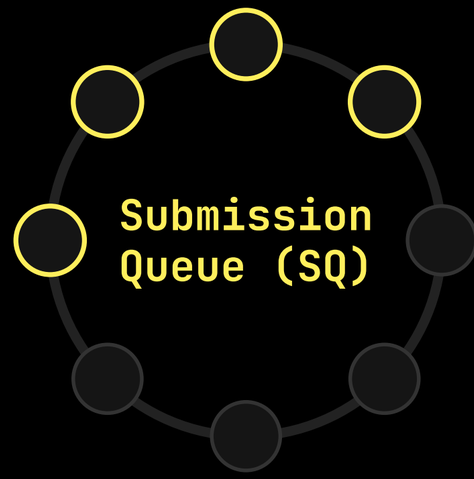


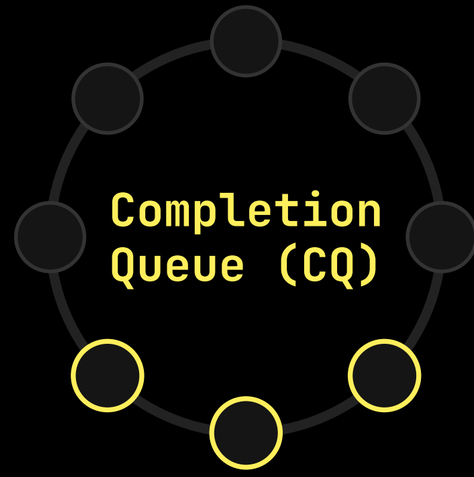
Async I/O: IO_URING

Async I/O



User | Kernel



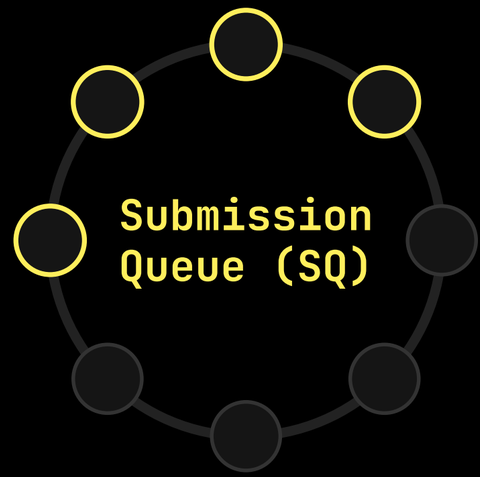


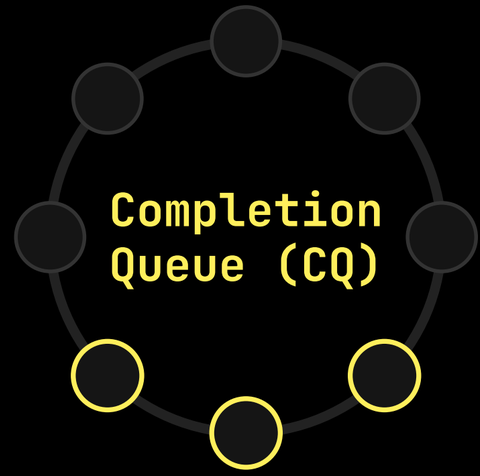
Async I/O: IO_URING

Async I/O



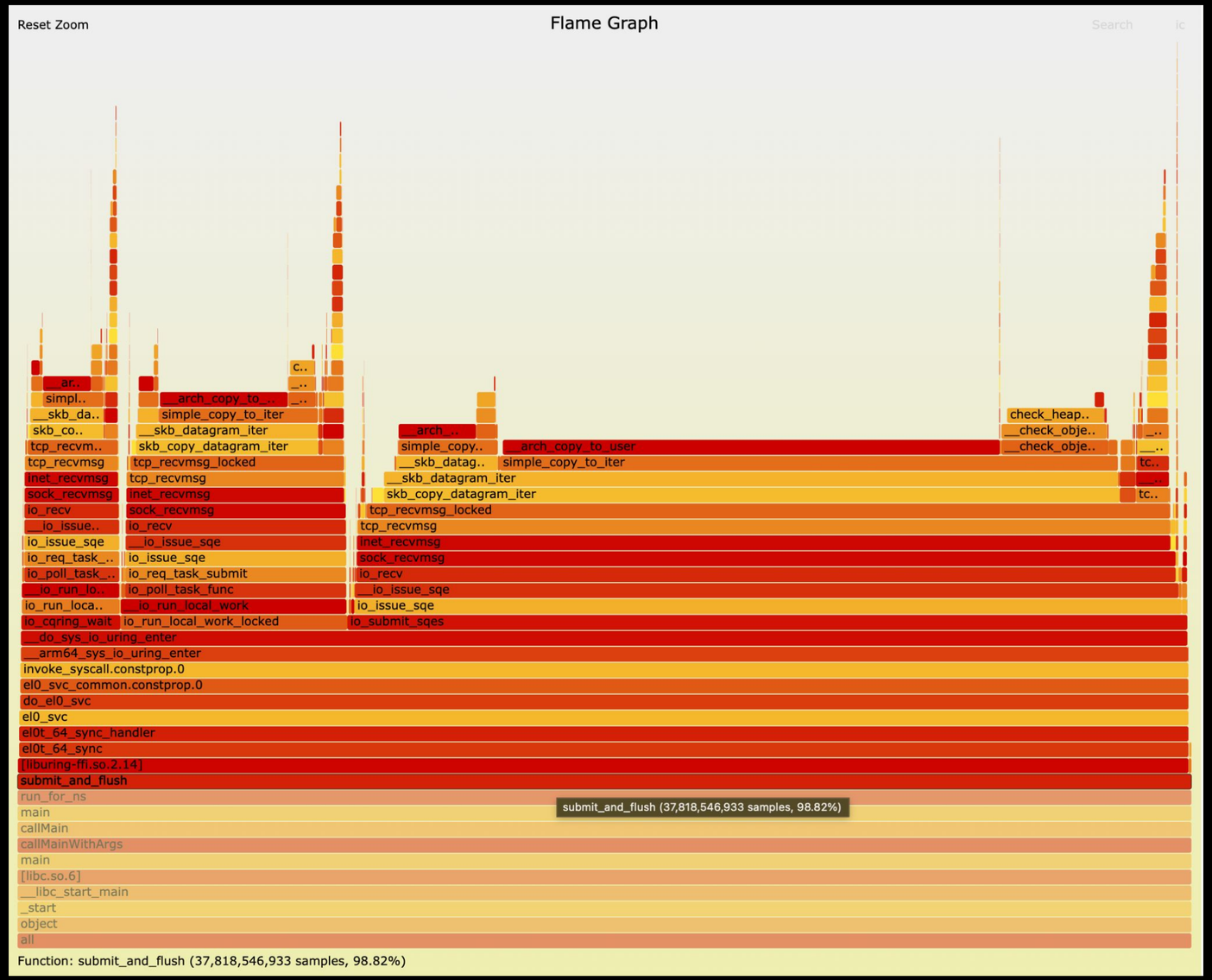
User | Kernel



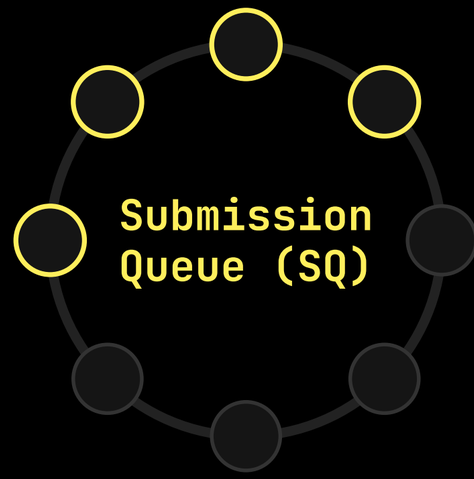


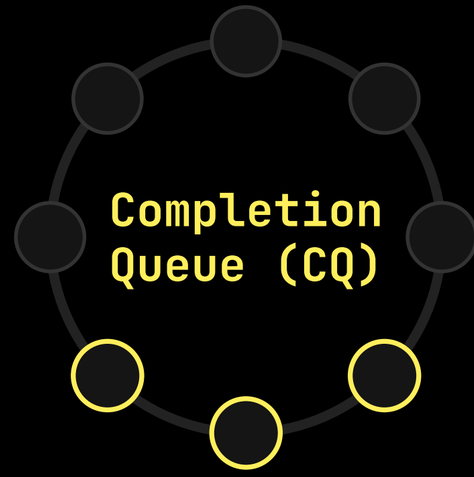
Async I/O: IO_URING

Async I/O



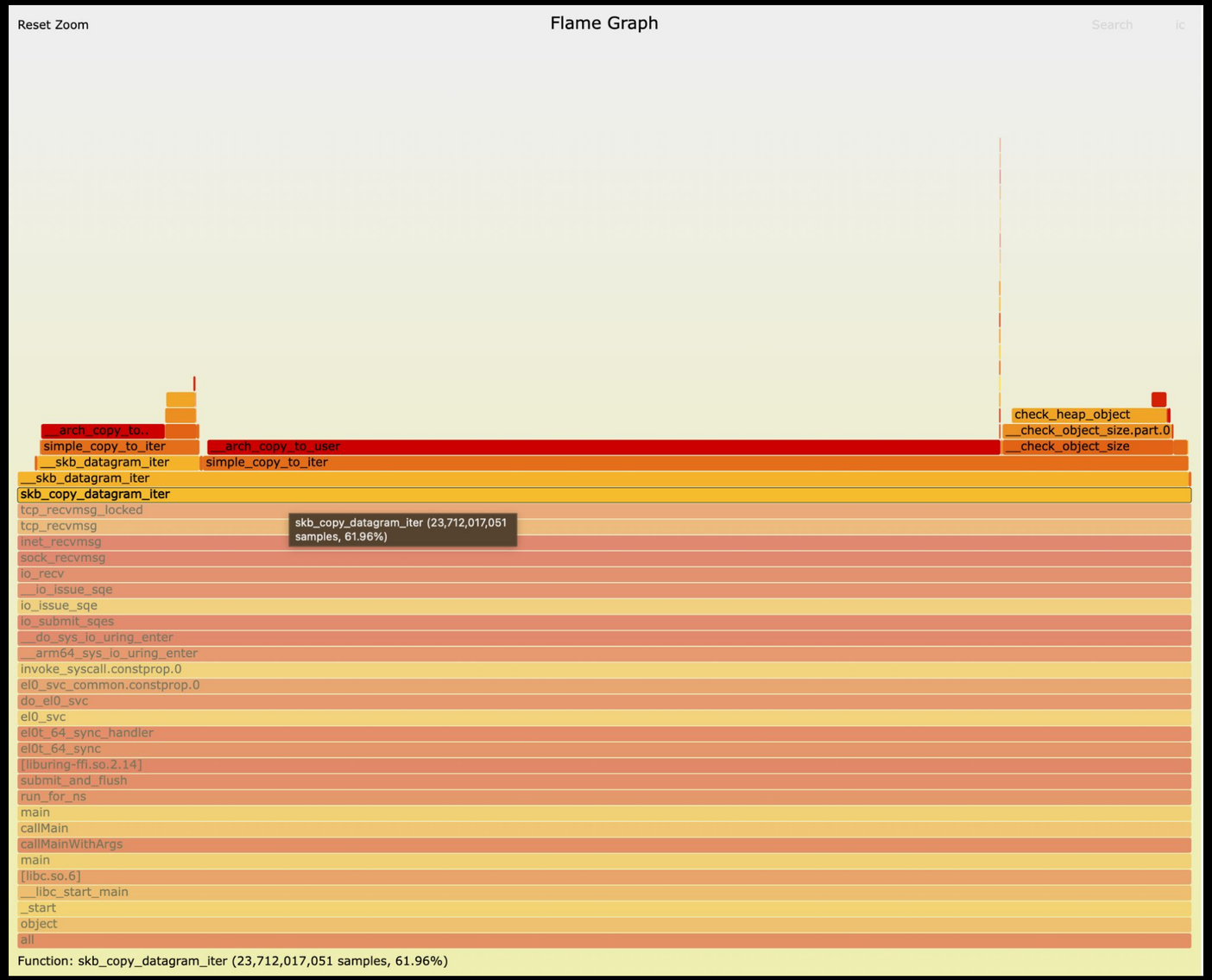
User | Kernel



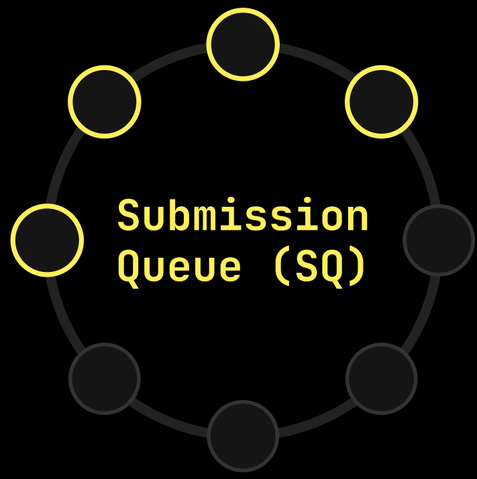


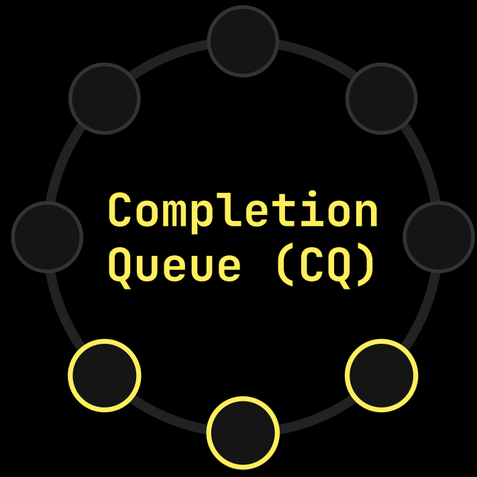
Async I/O: IO_URING

Async I/O



User | Kernel





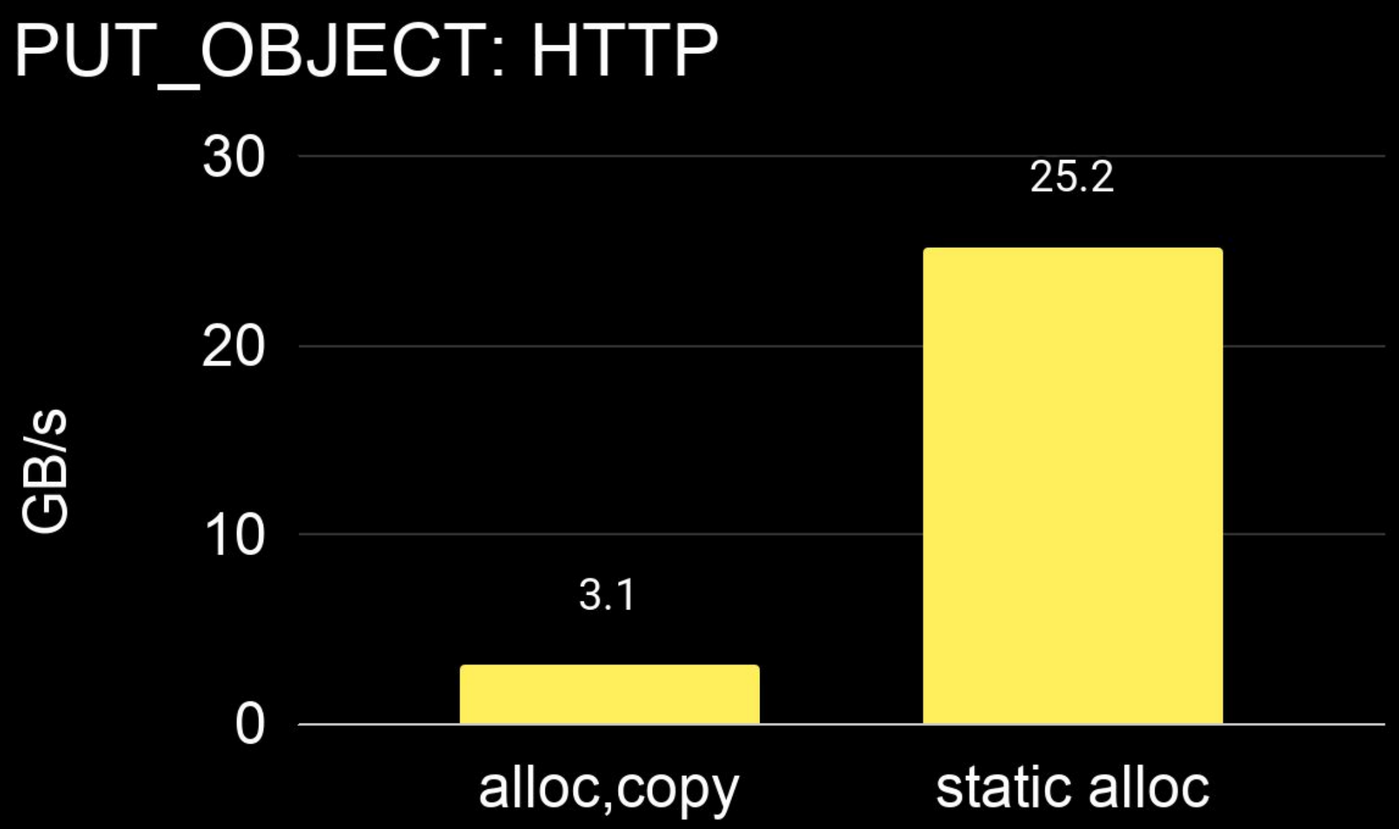
Hypertext Transfer Protocol

HTTP

- HTTP/1.0 and HTTP/1.1 support
- No allocation or copies on request path (compared to curl)
  - recv to / send from caller’s buffer
- Synchronous DNS lookup from in-memory cache
- Sans-I/O parsing implementation, heavily fuzzed
- Connections continuously reestablished
- HTTP server implementation for testing

Hypertext Transfer Protocol

HTTP



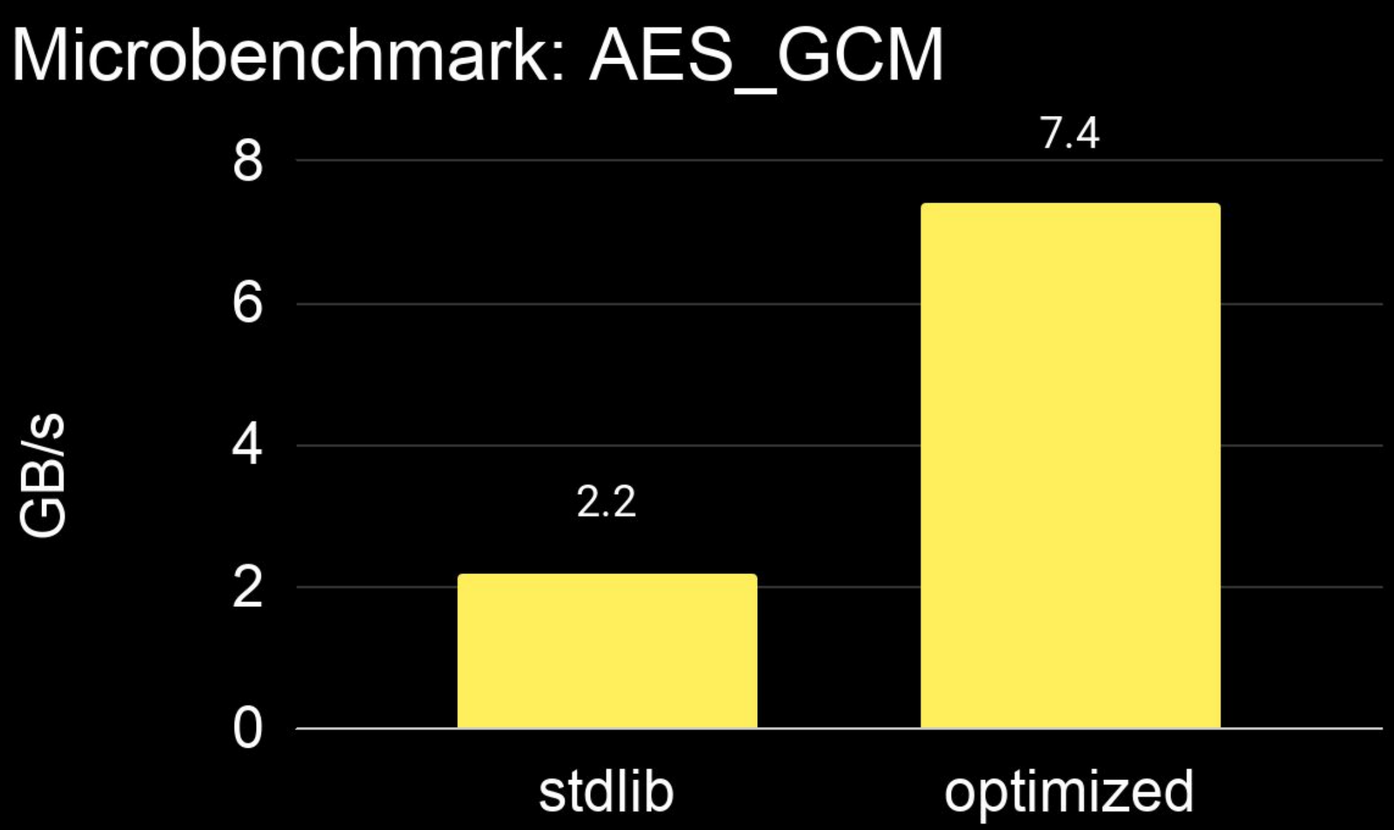
Transport Layer Security

TLS

- TLS 1.3 support
- Sans-I/O implementation, composable with async I/O
- Encryption buffer registered with kernel using zc send
- Optimized AES_GCM implementation
- Adapted interface `Encrypt` -> `EncryptWithSuffix`
- TLS Server implementation for testing

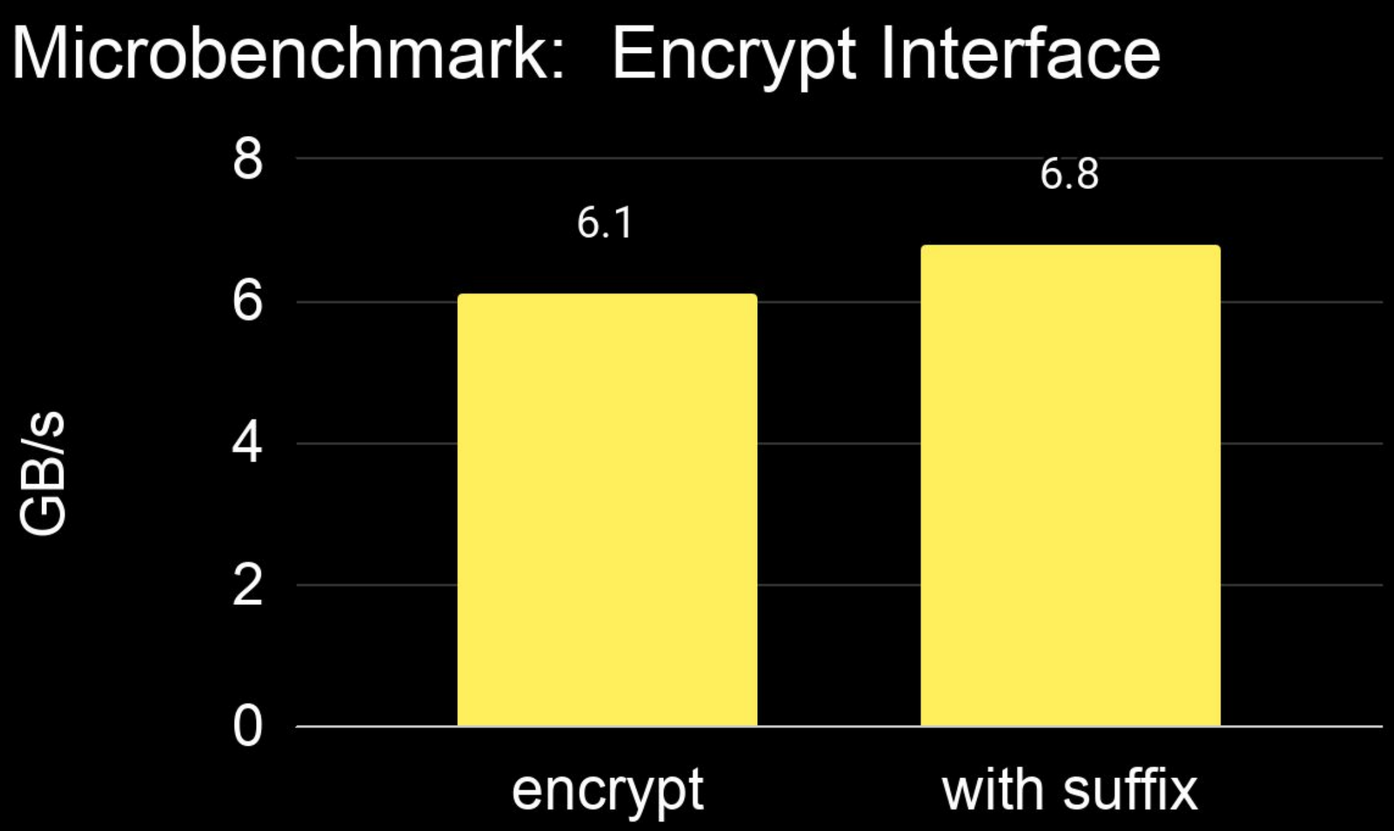
Transport Layer Security

TLS



Transport Layer Security

TLS



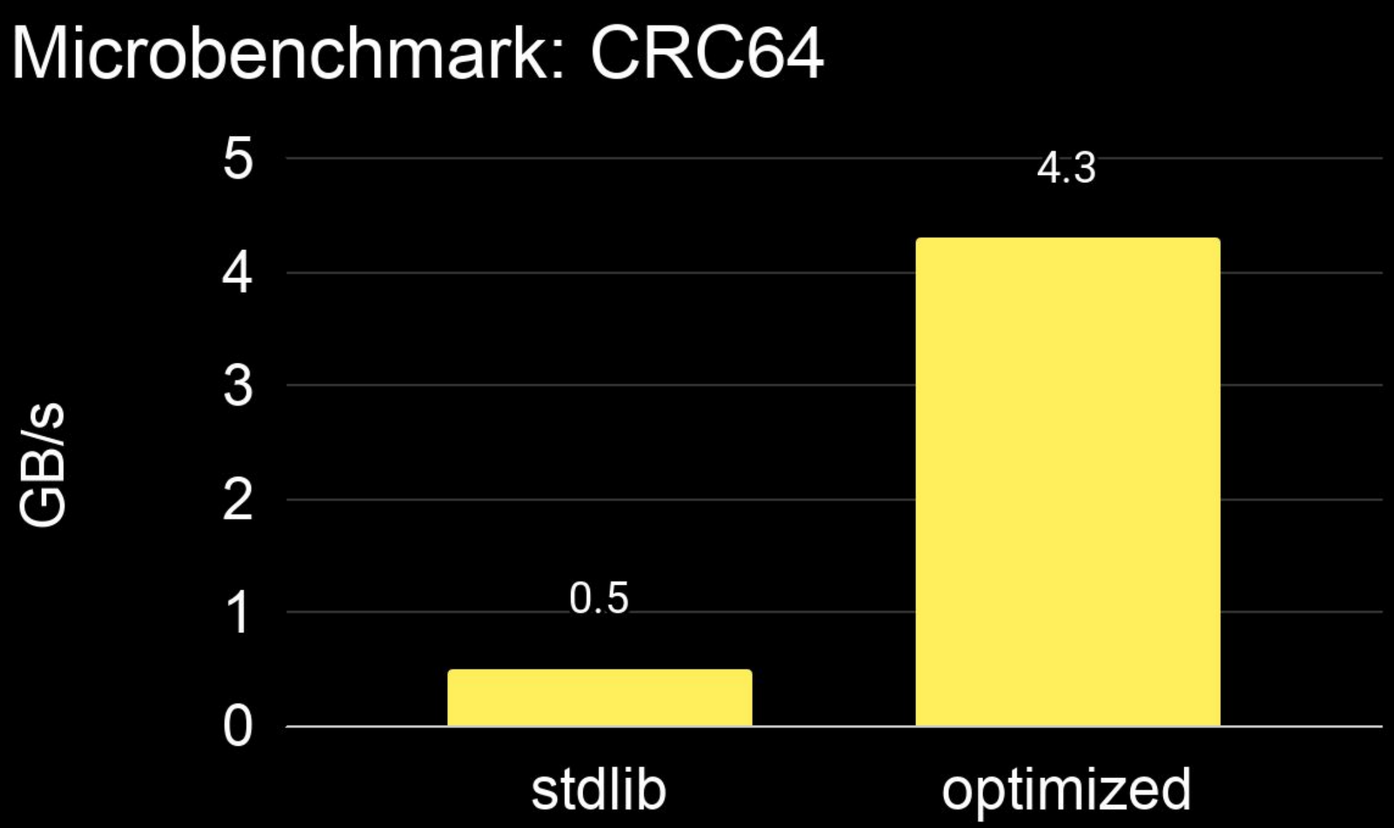
Object Storage API

S3 / Azure / GCP

- Unified API: `fetch`, `store`, `store_no_clobber`, `list`, `erase`
- Data integrity checks: CRC64-NVME (AWS, Azure), CRC32 (GCP)
- Graceful error handling: retryable, delayed (503), fatal
- Credentials Provider
  - Continuous reading and validation of credentials file
  - Validated by performing list request against object storage
  - Credentials hot-swap without dropping in-flight requests

Object Storage API

S3 / Azure / GCP



Summary



- Diagonal Scaling: extend the memory hierarchy with object storage
- Manage complexity by implementing only what you need (<40k LOC)
- Measure -> find bottleneck -> improve
- Deterministic Simulation Testing and Fuzzing
- Owning your entire stack is worth it
- If you do it right, it’s investment, not debt